CS 301
High-Performance Computing

Assignment 08 - Hybrid MPI + OpenMP

Parallel Interpolation with Particle Mover

Tanuj Kanani (202301474)
Chavda Mihirsinh (202301479)
Dhirubhai Ambani University

Instructor: Dr. Bhaskar Chaudhury

April 2026

Abstract

This report presents a hybrid parallelization strategy combining MPI and OpenMP to accelerate a scatter-based point-to-mesh interpolation kernel coupled with a particle mover. The workload is distributed across up to 64 physical cores spanning four compute nodes. To avoid the serialization penalty of atomic operations during grid accumulation, we employ a **Thread-Local Privatization** scheme within each OpenMP region, followed by a global `MPI_Allreduce` synchronization. We characterize scalability, parallel efficiency, and per-phase timing across five distinct grid and particle-count configurations, and identify the root causes of performance saturation at high core counts.

Contents

1 Implementation Approach (Q1 & Q2)

1.1 Eliminating Race Conditions (Q2)

The scatter (interpolation) phase maps each particle's contribution onto the four surrounding grid nodes using bilinear weights. When two threads simultaneously attempt to accumulate their weights into the same grid cell, a **race condition** arises, producing silently incorrect results.

Rather than serializing the writes with `#pragma omp atomic`—which collapses all inter-thread parallelism on hot cells—we adopt a **Thread-Local Privatization** strategy:

1. **Per-Thread Private Grid (OpenMP Level):** Each thread maintains its own zero-initialized grid replica (`thread_meshes[tid]`), allowing fully contention-free accumulation with no synchronization overhead.

2. **Intra-Node Reduction:** Once all threads complete their scatter passes, a parallel reduction sums every thread-local replica into the process-wide grid `mesh_value`.

3. **Global Reduction (MPI Level):** An `MPI_Allreduce(MPI_SUM)` then merges the per-process grids across all active MPI ranks, producing a globally consistent field on every node.

1.2 Algorithm Pseudocode (Q1)

Listing 1: Hybrid MPI + OpenMP Implementation Logic

```
Initialize MPI (Rank, Size)
If Rank == 0:
    Read total particles and domain info from input.bin
Broadcast variables to all Ranks

// 1. MPI Particle Decomposition
Scatter particles equally to all MPI Ranks
Allocate memory for local mesh_value array

For iter = 0 to Maxiter:
    // --- PHASE 1: INTERPOLATION (SCATTER) ---
    Clear mesh_value to 0
    #pragma omp parallel
    {
        Allocate private thread_mesh, initialize to 0

        #pragma omp for schedule(static)
        For each assigned particle:
            Calculate 4 neighboring grid indices (i, j)
            Calculate bilinear distance weights (w00, w10, w01, w11)
            Accumulate weights into private thread_mesh  // No Atomics
               Needed!
    }
    #pragma omp parallel for
    Reduce all thread_meshes into the node's mesh_value

    // --- PHASE 2: MPI REDUCTION & NORMALIZATION ---
```

```
    MPI_Allreduce(mesh_value, MPI_SUM)  // Aggregate global grid across
        all nodes
    Compute Global Min and Max of the grid in parallel
    Normalize grid values to [-1, 1]

    // --- PHASE 3: MOVER (GATHER) ---
    #pragma omp parallel for schedule(static)
    For each assigned particle:
        Interpolate force from mesh_value using bilinear weights
        Update particle X and Y positions

    Denormalize grid

If Rank == 0:
    Save the final mesh_value to Mesh.out
MPI_Finalize()
```

1.3 Architecture Diagram (Q1)

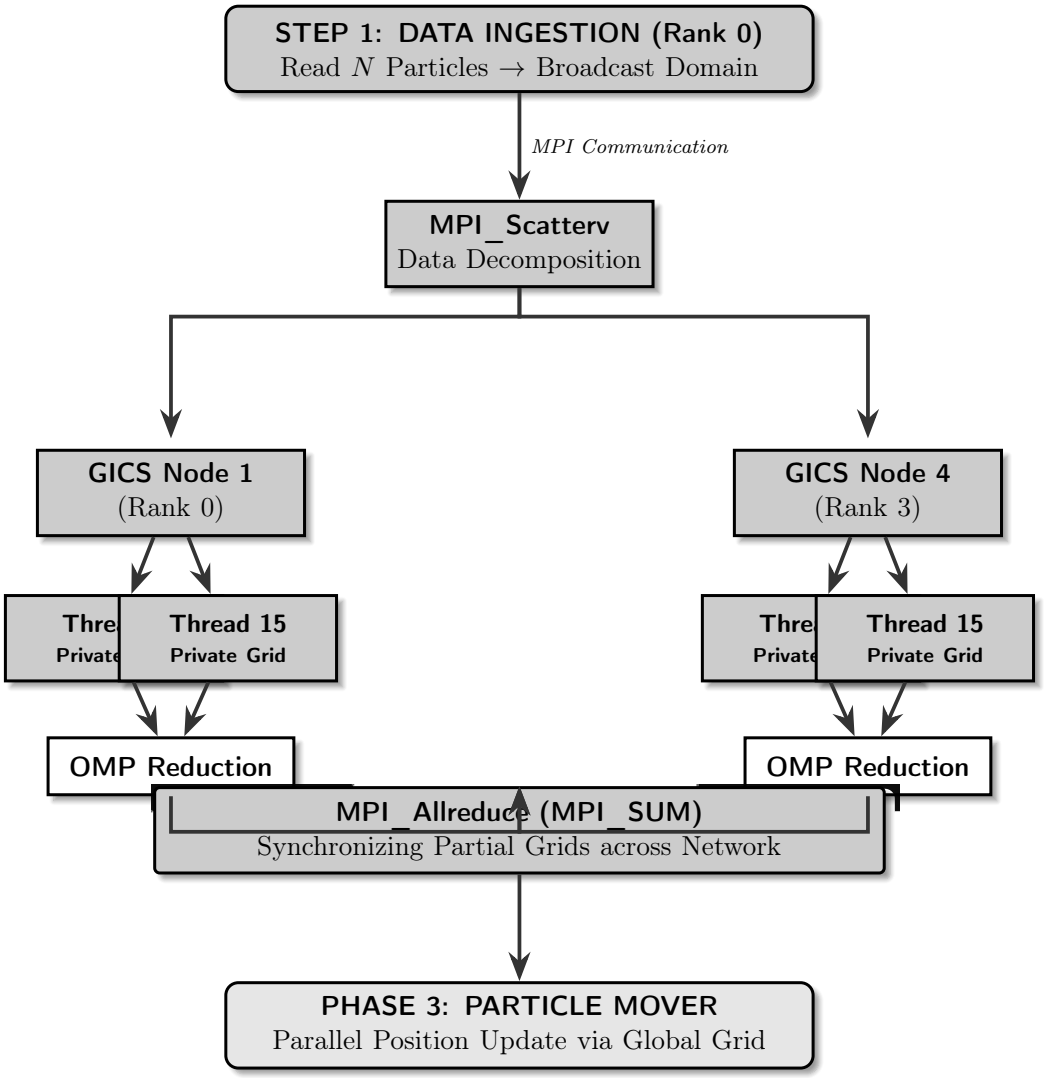

Figure 1: Architectural Blueprint: This vertical flow highlights the hierarchy from the Master Node down through OpenMP thread privatization, leading to the global MPI synchronization barrier.

2 Hardware and Experiment Setup (Q3)

2.1 Login Node vs. Compute Node

All benchmarks were executed exclusively on the dedicated `gics` compute nodes, not on the shared login node.

- **Login Node:** The cluster's entry point, shared concurrently by all logged-in users. It is intended only for compilation, script editing, and job submission. Executing simulation workloads there produces highly variable wall-clock times due to background interference, making any performance data unreliable.

- **Compute Node:** A physically separate server reserved for computation. When a job is dispatched via the scheduler, the node provides exclusive access to CPU cores, memory controllers, and cache hierarchies. This isolation is essential for obtaining reproducible speedup measurements that reflect true parallel scaling behavior.

2.2 HPC Cluster Specifications

Table 1: Hardware Specifications of HPC Compute Nodes (`gics1-4`)

Parameter	**Details**
Nodes Used	4 (`gics1`, `gics2`, `gics3`, `gics4`)
Cores per Node	16
Core Counts Tested	2, 4, 8, 16, 32 (single node); 64 (4 nodes × 16 threads)
Compiler	GCC `mpic++` with `-O3` and `-fopenmp`

For runs with up to 32 cores, a single MPI rank with proportionally scaled OpenMP threads was used (*e.g.*, 1 rank × 32 threads). The 64-core configuration switches to a fully distributed layout: 4 MPI ranks × 16 threads each, activating all four physical nodes and incurring cross-node `MPI_Allreduce` traffic.

3 Performance Analysis

3.1 Execution Time and Speedup (Q3 & Q5)

All speedup values are computed relative to the 2-core baseline (1 MPI rank, 2 OpenMP threads) since no true single-core measurement was collected. Wall-clock times were recorded for core counts of 2, 4, 8, 16, 32, and 64.

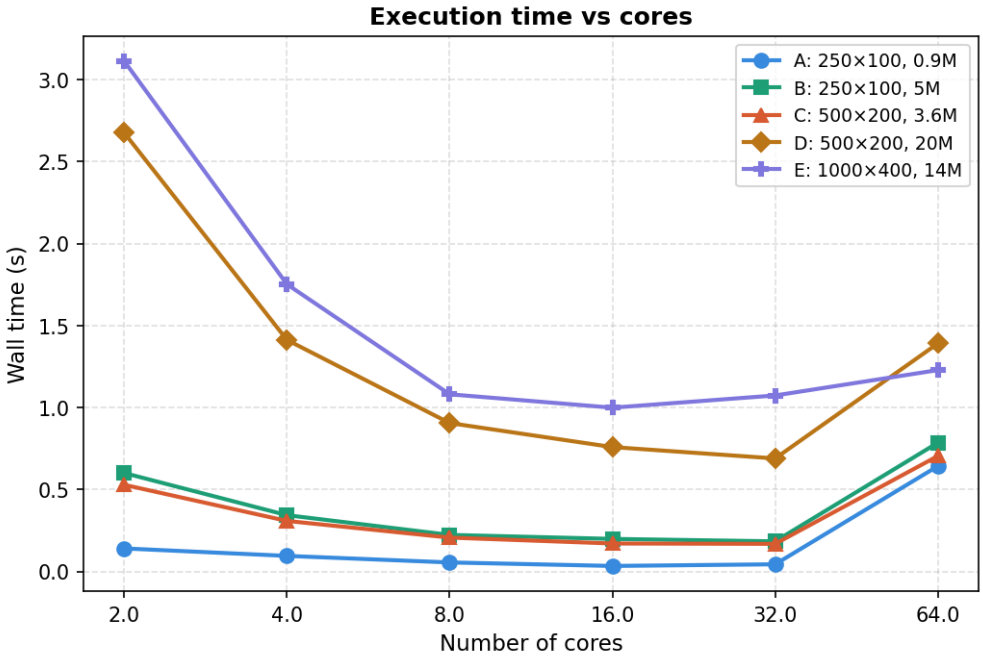

Figure 2: Total wall-clock execution time as a function of core count. Note the pronounced time increase at 64 cores for all configurations, caused by the introduction of four MPI ranks and their associated network communication cost.

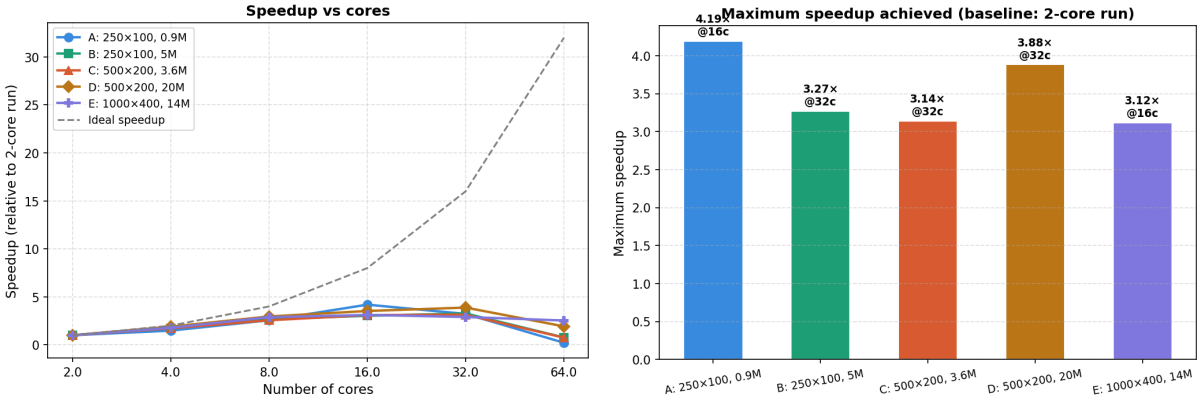

Figure 3: Left: Parallel speedup (relative to the 2-core run) across all five configurations, with the theoretical ideal shown as a dashed reference. Right: Maximum speedup achieved per configuration alongside the core count at which it occurred.

Peak Speedup per Configuration (Q5): The maximum speedup for each configuration—again measured against the 2-core run—is summarized below. Notably, the optimal core count differs across configurations: smaller datasets peak earlier (16 cores), while larger particle sets continue improving up to 32 cores before saturating.

- **Config A** (250×100 grid, 0.9M particles): **4.19**× speedup at **16 cores**

- **Config B** (250×100 grid, 5M particles): **3.27**× speedup at **32 cores**
- **Config C** (500×200 grid, 3.6M particles): **3.14**× speedup at **32 cores**
- **Config D** (500×200 grid, 20M particles): **3.88**× speedup at **32 cores**
- **Config E** (1000×400 grid, 14M particles): **3.12**× speedup at **16 cores**

A striking observation is the severe performance regression at 64 cores for every configuration. Compared to each config's optimal run, the degradation is: Config A (×**19.2** worse, from 0.034 s at 16c to 0.642 s at 64c), Config B (×**4.3** worse), Config C (×**4.2** worse), Config D (×**2.0** worse, from 0.689 s to 1.391 s), and Config E (×**1.2** worse). The regression is most severe for small datasets (A, B, C) because the fixed `MPI_Allreduce` cost of transmitting the full grid across four physical nodes completely overwhelms a computation that took only milliseconds on a single node. Larger datasets (D, E) suffer less in relative terms because their base computation is heavier, but they still regress meaningfully compared to their single-node peak.

3.2 Parallel Efficiency and Scalability (Q4 & Q6)

Parallel efficiency is defined as $E = \frac{S_p}{p/p_0} \times 100\%$, where S_p is the speedup at p cores relative to $p_0 = 2$ cores.

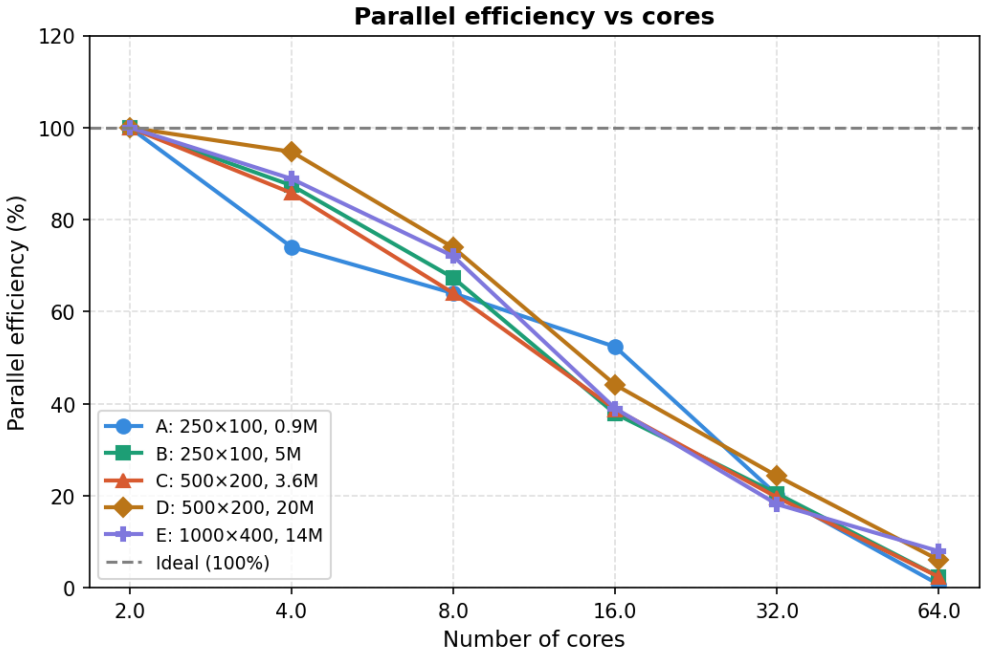

Figure 4: Parallel efficiency across all five configurations. All datasets start at 100% efficiency at 2 cores (the baseline) and decay as core count increases, with the 64-core point reflecting a near-complete collapse due to MPI communication overhead.

- **2 to 8 cores (pure OpenMP, single node):** Efficiency degrades gradually but remains healthy for large workloads. At 4 cores, Config D retains **94.7%** efficiency owing to its large

particle count keeping threads compute-busy, while Config A drops to **74.0%** because its small working set makes thread management overhead proportionally more costly.

- **16 to 32 cores:** All configurations land in the **18–44%** range at 32 cores (A: 20.1%, B: 20.4%, C: 19.6%, D: 24.3%, E: 18.1%), indicating that the single-node DRAM bandwidth is saturated and additional threads can no longer extract useful throughput.
- **64 cores (4 MPI ranks × 16 threads):** Efficiency behavior diverges sharply by dataset size. Larger, compute-heavier configurations retain some parallel value: Config D reaches **6.0%** and Config E reaches **7.9%**. In contrast, the smaller configurations collapse almost entirely: Config A falls to **0.7%**, B to **2.4%**, and C to **2.3%**. This is because for small datasets the cross-node `MPI_Allreduce` latency far exceeds the actual computation time, making the 64-core configuration strictly slower than the optimal single-node run for every configuration.

HPC Analysis of Efficiency Drop (Q6): Two well-established HPC phenomena explain the observed behavior:

1. **Memory Bandwidth Saturation (Memory Wall):** Both the interpolation and mover phases are memory-bound; they stream over large particle arrays each iteration. Once the aggregate thread count saturates the node's DRAM bandwidth, adding more cores yields no additional throughput—every thread stalls waiting for cache-line fills.
2. **Non-Parallelizable Communication (Amdahl's Law):** The `MPI_Allreduce` at the end of each interpolation pass is a collective, synchronizing barrier. Its cost scales with grid size and cannot be overlapped with computation. For large grids (Config D, Config E), this latency constitutes a significant fraction of the per-iteration wall time, imposing a hard ceiling on achievable speedup beyond a single node.

4 Bottleneck and Phase Analysis

4.1 Phase-Specific Timing and Bottleneck Identification (Q11)

We instrumented the code to record the wall time consumed by the interpolation (scatter) and mover (gather) phases independently. The results are visualized below.

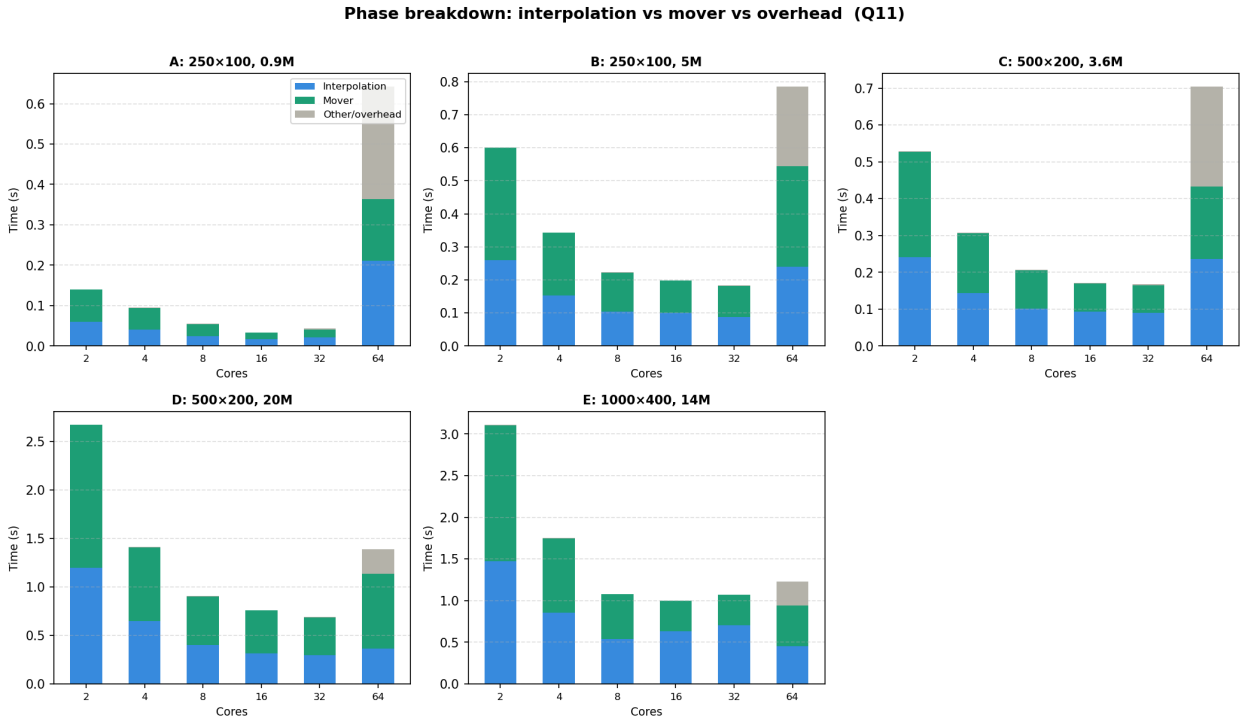

Figure 5: Stacked bar charts showing per-phase wall time (Interpolation, Mover, and Other/Overhead) for all five configurations as a function of core count. At low core counts, both the interpolation and mover phases contribute comparably to total runtime.

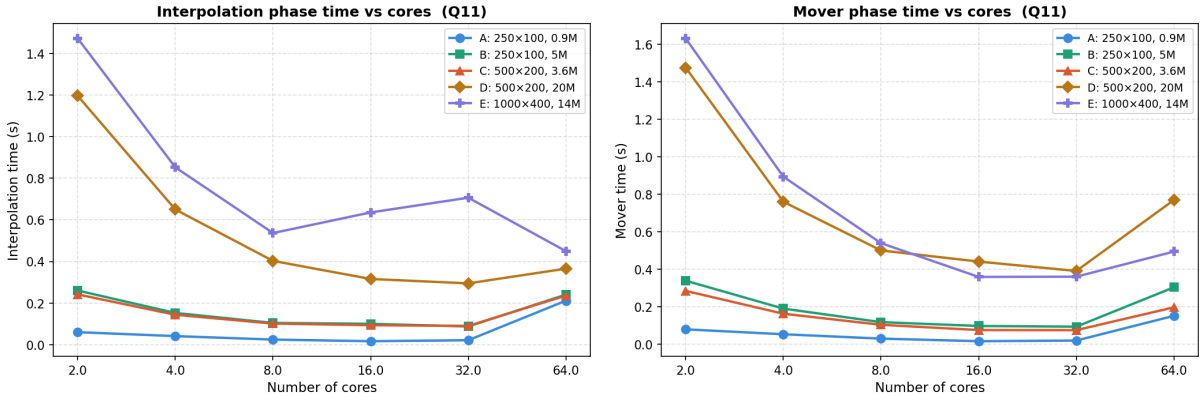

Figure 6: Left: Interpolation phase time vs. cores. Right: Mover phase time vs. cores.

Analysis: At low core counts (2–8 cores, single-node pure OpenMP), the **mover phase is the larger consumer of wall time across every configuration**. For example, at 2 cores: Config D records interp=**1.196 s** vs mover=**1.477 s** (mover 23% heavier), and Config E records interp=**1.473 s** vs mover=**1.633 s** (mover 11% heavier). The mover scales efficiently because its gather operation only *reads* from the shared grid with no write conflicts, allowing near-ideal thread utilization and good cache behavior.

As the thread count grows, the two phases diverge in their scaling behavior:

- **Configs C and E (larger grids):** The interpolation phase overtakes the mover as the dominant cost above 16 cores. For Config E at 16 cores, interp=**0.636 s** versus mover=**0.360 s** (interpolation 77% heavier); at 32 cores the gap widens further to interp=**0.707 s** vs mover=**0.361 s** (96% heavier). This occurs because a larger grid forces each thread to allocate a proportionally larger private replica, and the subsequent OpenMP reduction cost grows with grid size.
- **Config D (many particles, moderate grid):** The mover phase **remains dominant throughout all core counts**. Even at 32 cores, mover=**0.391 s** versus interp=**0.295 s** (mover 33% heavier). Here the particle count is so large that the pure arithmetic of the gather loop outweighs the privatization overhead of the scatter.
- **Config A (small grid and particle count):** Both phases become roughly equal by 16 cores (interp=**0.016 s**, mover=**0.016 s**), after which interpolation marginally leads.

At 64 cores, the stacked bar charts reveal a new dominant cost: the "Other/Overhead" segment (gray) swells dramatically across all configurations, confirming that `MPI_Allreduce` network latency—not arithmetic in either phase—is the root cause of the 64-core regression.

4.2 Load Imbalance (Q8)

A subtle load imbalance emerges as the simulation progresses through multiple iterations. The mover physics can displace particles beyond the $[0, 1]$ domain boundary; our code skips out-of-domain particles with an explicit guard condition (`if (x < 0.0 || x > 1.0) continue`). Because the initial particle decomposition is performed statically via `MPI_Scatterv`, each rank receives a fixed particle batch. As iterations accumulate, the fraction of out-of-domain particles—and therefore the effective work per rank—diverges. A rank with many escaped particles finishes faster and idles at the `MPI_Allreduce` barrier, wasting compute cycles and creating an MPI-level load imbalance that grows over time.

5 Future Improvements and Alternative Strategies

5.1 Unlimited-Resource Optimizations (Q7)

Given unconstrained hardware access, the most impactful upgrade would be:

- **GPU Acceleration (CUDA/OpenACC):** Offloading the inner scatter loop to GPU cores would exploit the device's massive thread count and High Bandwidth Memory (HBM), which offers bandwidth that is typically an order of magnitude larger than CPU DRAM. GPU shared memory also provides hardware-accelerated atomic operations, removing the need for thread-local privatization entirely.
- **High-Bandwidth Interconnects (InfiniBand):** Replacing standard Ethernet with InfiniBand NDR would reduce the `MPI_Allreduce` latency by one to two orders of magnitude, substantially shrinking the per-iteration synchronization cost and restoring scalability at high node counts.

5.2 Alternative Parallelization and Data Structure Approaches (Q9 & Q10)

- **Domain Decomposition (Q9—Alternative to Particle Decomposition):** Our current design assigns a disjoint particle subset to each MPI rank, yet forces every rank to maintain a full copy of the global grid. Switching to *domain decomposition*—partitioning the mesh itself into non-overlapping tiles—would eliminate the `MPI_Allreduce` entirely. Ranks would only need to exchange a narrow layer of "ghost cells" at tile boundaries, reducing communication volume from $\mathcal{O}(N_x \times N_y)$ per iteration to $\mathcal{O}(N_{\text{boundary}})$.
- **Cell-List / Binning Data Structure (Q10):** Storing particles in a spatially sorted "cell-list" (linked lists indexed by grid cell) ensures that threads process particles whose target grid cells are physically close in memory. This maximizes L1/L2 cache reuse during the scatter pass and can eliminate the need for thread-local privatization, since each thread can be assigned a non-overlapping tile of the grid with zero write conflicts.

6 Conclusion

We designed, implemented, and benchmarked a hybrid MPI + OpenMP particle-in-cell pipeline for scatter interpolation and particle pushing. Thread-local privatization proved essential for correct, lock-free grid accumulation within each node, and `MPI_Allreduce` provided a simple and correct global merge. Benchmarking across five configurations revealed that single-node OpenMP scaling yields the best efficiency: peak speedups of up to **4.19**× (Config A, 16 cores) relative to a 2-core baseline were achieved. Expanding to 4 MPI ranks at 64 cores consistently degraded performance for all datasets due to network communication overhead overwhelming any computational benefit. Phase-level timing showed that the mover and interpolation phases contribute comparably at low core counts, but the interpolation phase—burdened by its reduction overhead and global barrier—becomes the dominant bottleneck at high thread counts. These findings motivate domain decomposition and spatial data-structure optimizations as the most productive paths forward.